

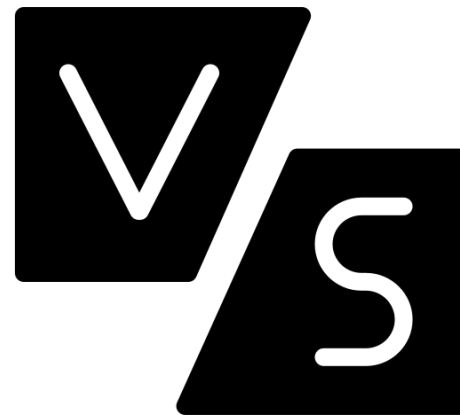
Programmazione Web e Mobile con Laboratorio

Lezione 17 - Computazioni pesanti
(Web Workers & worker_threads)

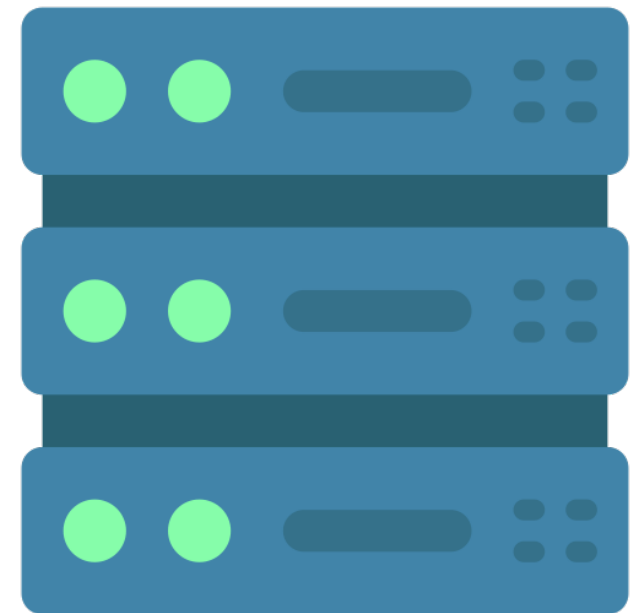
Computazioni lunghe



CLIENT



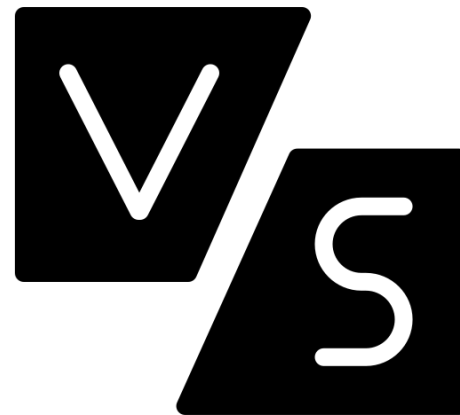
SERVER



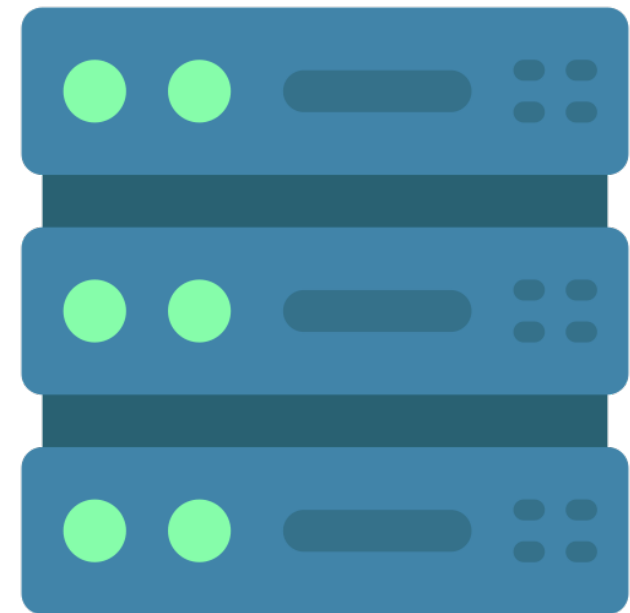
Computazioni lunghe



CLIENT



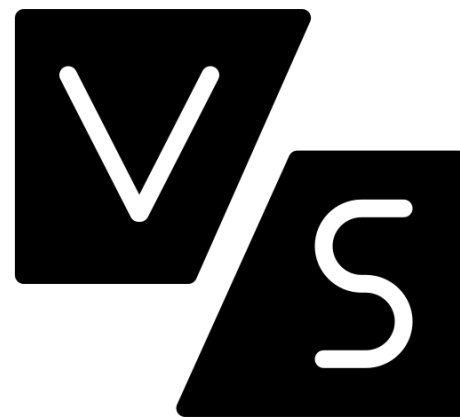
SERVER



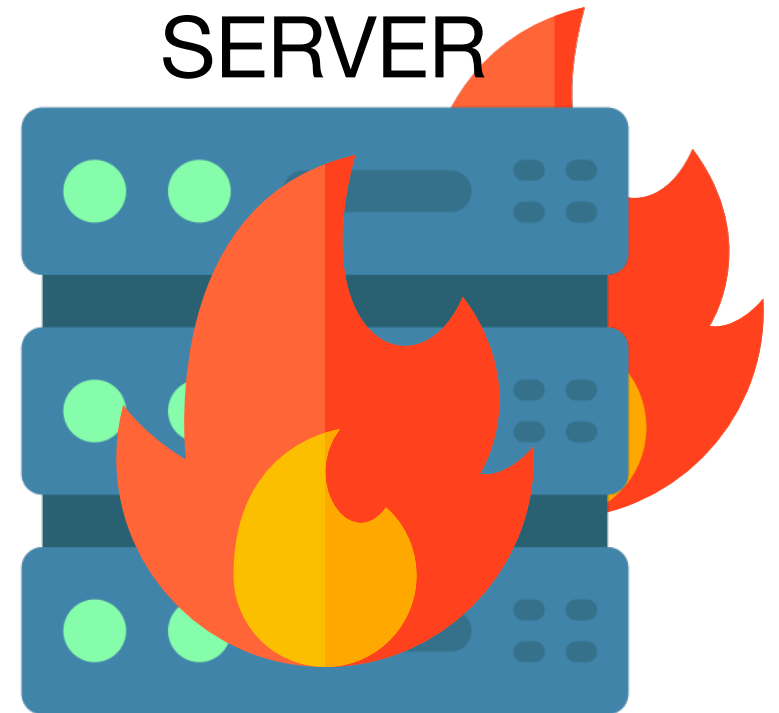
Computazioni lunghe



CLIENT



SERVER



Client vs server

- **Lato client**
- Tutto avviene nel browser, quindi non c'è latenza di rete
- Ogni utente esegue la propria computazione e non carica il server
- I dati non lasciano il client, ottimo per la privacy
- Esempi tipici:
 - Compressione immagini, parsing di file grandi, crittografia
 - Elaborazione di file caricati dall'utente (CSV, JSON, immagini)
 - Rendering canvas pesante (grafica, giochi)
 - Calcoli scientifici, ML in browser (es. TensorFlow.js)

Client vs server

- **Lato client**

- Tutto avviene nel browser, quindi non c'è latenza di rete
- Ogni utente esegue la propria computazione e non carica il server
- I dati non lasciano il client, ottimo per la privacy
- Esempi tipici:
 - Compressione immagini, parsing di file grandi, crittografia
 - Elaborazione di file caricati dall'utente (CSV, JSON, immagini)
 - Rendering canvas pesante (grafica, giochi)
 - Calcoli scientifici, ML in browser (es. TensorFlow.js)

- **Lato server**

- Il client resta leggero (per dispositivi con poca potenza)
- Logica centralizzazione utile per accedere a database, filesystem, API interne, ecc.
- Computazioni pesanti con GPU o CPU dedicate
- Esempi tipici:
 - ML lato server
 - Analisi dati su grandi dataset condivisi
 - Elaborazioni asincrone con coda (es. video encoding)
 - Logica business sensibile da non rivelare

**Lato client:
Web Workers**

Esempio calcolo su client

- Calcolare i numeri primi fino a 20 milioni

```
function isPrime(n) {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i == 0) return false;
  }
  return true;
}

function startComputation() {
  ...
  const primes = [];
  const max = 20000000;
  for (let i = 0; i < max; i++) {
    if (isPrime(i)) { primes.push(i); }
    if (i % 100000 == 0) { progress.value = (i / max) * 100; }
  }
  progress.value = 100;
  resP.textContent = "Numeri primi trovati: " + primes.length;
}
```


Esempio calcolo su client

- Calcolare i numeri primi fino a 20 milioni
- Problema: la computazioni blocca il client

```
function isPrime(n) {  
  if (n < 2) return false;  
  for (let i = 2; i <= Math.sqrt(n); i++) {  
    if (n % i == 0) return false;  
  }  
  return true;  
}  
  
function startComputation() {  
  ...  
  const primes = [];  
  const max = 20000000;  
  for (let i = 0; i < max; i++) {  
    if (isPrime(i)) { primes.push(i); }  
    if (i % 100000 == 0) { progress.value = (i / max) * 100; }  
  }  
  progress.value = 100;  
  resP.textContent = "Numeri primi trovati: " + primes.length;  
}
```



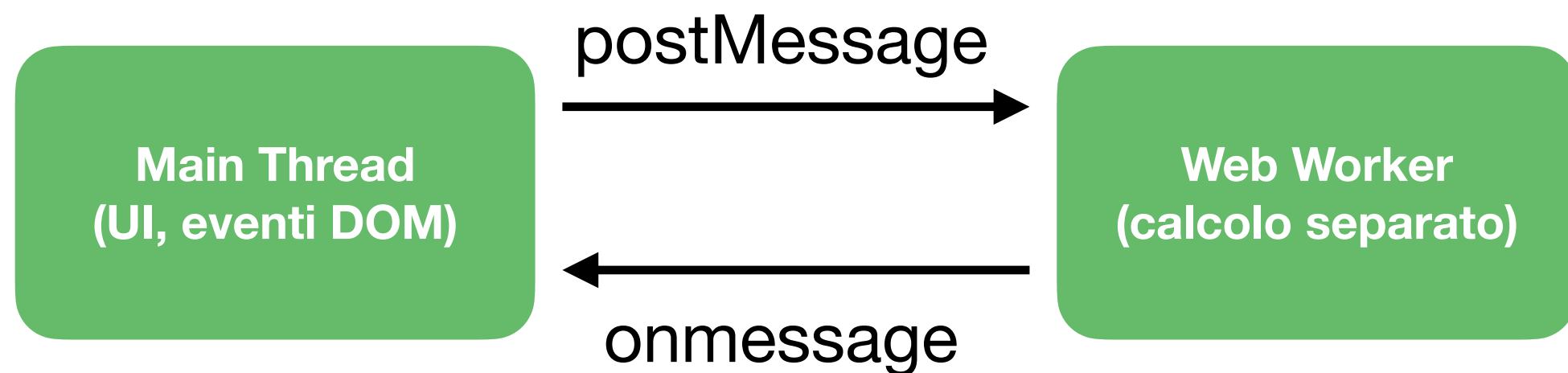
Web Workers in JavaScript

- Le computazioni pesanti bloccano il thread principale del browser (UI non reattiva, freeze, brutta esperienza utente)
- I Web Workers consentono di eseguire codice JavaScript in un thread separato
- Perfetti per calcoli intensivi senza bloccare la UI
- Non possono essere caricati da locale, ma richiedono sempre un server HTTP



Architettura

- Comunicazione bidirezionale tramite messaggi (no shared memory)
- Il Web Worker (es. un file worker.js) contiene il codice che verrà eseguito nel thread separato



Main thread

```
const worker = new Worker("script/worker.js");
worker.postMessage({ max: 20000000 });

worker.onmessage = (event) => {
  if (event.data.type === "progress") {
    progress.value = e.data.value;
  }
  else if (event.data.type === "done") {
    progress.value = 100;
    resP.textContent = "Numeri primi trovati: " + e.data.count;
    worker.terminate();
  }
};
```

- `worker.postMessage` invia un oggetto (dato serializzabile)
- `worker.onmessage` ascolta i messaggi inviati dal worker e `event.data` contiene il messaggio
- `worker.terminate()` termina il worker

Worker thread

```
onmessage = function (e) {  
  const max = e.data.max;  
  const primes = [];  
  
  for (let i = 0; i < max; i++) {  
    if (isPrime(i)) primes.push(i);  
  
    if (i % 100000 == 0) {  
      postMessage({ type: "progress", value: (i / max) * 100 });  
    }  
  }  
  
  postMessage({ type: "done", count: primes.length });  
};
```

- Il worker riceve i dati tramite `onmessage`
- `postMessage()` invia i dati processati al main thread

Interrompere il worker

- Quando si chiude la scheda del browser, il main thread viene terminato e con lui tutti i Web Worker associati
- Chiamare `worker.terminate()` dal main thread
- Chiamare `self.close()` all'interno del worker
- Temporizzare la terminazione con `setTimeout()`
- Non si può mettere in pausa un worker e riprendere in seguito

Lato server:
worker_threads

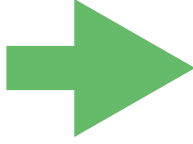
Computazioni lato server

- Eseguire computazioni pesanti
- Comunicare stato o risultato al client

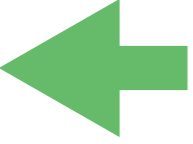
Tecnica	Comunicazione	Progressivo?	Interrompibile da client?	Complessità implementativa	Note
Fetch tradizionale	Richiesta/risposta HTTP	No	Solo su chiusura finestra browser	Bassa	Semplice, ma poco versatile
Polling	Richieste ripetute	Sì (a intervalli)	Possibile con gestione stato	Media	Richiede gestione stato e più richieste HTTP
Server-Sent Events	Connessione HTTP unidirezionale	Sì	Possibile con close event	Media	Aggiornamenti push, unidirezionale da server a client
WebSocket	Connessione full-duplex	Sì	Sì	Alta	Bidirezionale simultanea, ideale per aggiornamenti real-time
ReadableStream HTTP	Streaming risposta HTTP	Sì	No	Alta	Streaming dati in chunk

Computazioni lato server

- Eseguire computazioni pesanti
- Comunicare stato o risultato al client



Tecnica	Comunicazione	Progressivo?	Interrompibile da client?	Complessità implementativa	Note
Fetch tradizionale	Richiesta/risposta HTTP	No	Solo su chiusura finestra browser	Bassa	Semplice, ma poco versatile
Polling	Richieste ripetute	Sì (a intervalli)	Possibile con gestione stato	Media	Richiede gestione stato e più richieste HTTP
Server-Sent Events	Connessione HTTP unidirezionale	Sì	Possibile con close event	Media	Aggiornamenti push, unidirezionale da server a client
WebSocket	Connessione full-duplex	Sì	Sì	Alta	Bidirezionale simultanea, ideale per aggiornamenti real-time
ReadableStream HTTP	Streaming risposta HTTP	Sì	No	Alta	Streaming dati in chunk



worker_threads in Node.js

- Modulo nativo per eseguire thread paralleli
- Permette di fare computazioni pesanti senza bloccare l'event loop principale
- Il main thread crea uno o più Worker Threads
- Ogni Worker esegue uno script JS in un contesto separato
- Comunicazione tra main e worker tramite messaggi
`postMessage` / `on( "message" )`

require("worker_threads")

```
const { Worker, isMainThread, parentPort, workerData } = require("worker_threads");
```

- Worker: crea un nuovo thread eseguendo un file JS
- isMainThread: indica se il codice è eseguito nel main thread o nel worker
- parentPort: porta di comunicazione con il thread padre (disponibile solo per il worker)
- workerData: dati passati al worker all'avvio (solo per il worker)

Esempio

- Calcolare i primi n numeri primi
- Server.js

```
const { Worker } = require("worker_threads");
...
app.get("/calcola", (req, res) => {
  const n = parseInt(req.query.count);
  const worker = new Worker("./worker.js", { workerData: n });
  req.on("close", () => {
    console.log("Client disconnesso, termino il worker");
    worker.terminate();
  });
  worker.on("message", primi => {
    res.json({ primi });
  });
  worker.on("error", err => {
    console.error("Errore nel worker:", err);
    res.status(500).send("Errore nel worker: " + err.message);
  });
});
...
```

Worker nel server

- `new Worker("./worker.js", { workerData: n })` crea un nuovo worker thread eseguendo il file `worker.js` e passa il valore `n` come parametro
- `worker.on("message", primi => {...})` ascolta l'evento "message" inviato dal worker e invia `primi` al client
- `req.on("close", () => {...})` ascolta l'evento `close` sulla richiesta HTTP e termina il worker

Esempio (cont)

- Calcolare i primi n numeri primi
- Worker.js

```
const { parentPort, workerData } = require("worker_threads");
function calcolaPrimi(n) {
  const primi = [];
  for (let i = 2; primi.length < n; i++) {
    if (primi.every(p => i % p !== 0)) {
      primi.push(i);
    }
  }
  return primi;
}
const primi = calcolaPrimi(workerData);
parentPort.postMessage(primi);
```

- `parentPort.postMessage(primi)` invia il risultato al thread principale

Ricapitolando

- Il server riceve la richiesta e avvia un calcolo pesante in un worker separato
- Quando il calcolo finisce, il risultato viene inviato al client
- Se il client si disconnette, il worker viene terminato per evitare sprechi
- Gli errori nel worker vengono gestiti e comunicati